

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: 3

Student Name: K. Saranya

Roll No.: A24126552090

1. TITLE

Build an Interactive To-Do List Application Using JavaScript DOM Manipulation

2. AIM

To build an interactive to-do list web application that lets a user add, mark complete, and delete tasks dynamically using JavaScript DOM manipulation, without reloading the page.

3. OBJECTIVE

The objectives of this experiment are:

- To capture user input from a text field using the .value property.
- To validate input and alert the user when the task field is left empty.
- To dynamically create task elements using document.createElement().
- To append dynamically created elements to the page using appendChild().
- To handle click events on dynamically created buttons using addEventListener().
- To toggle a CSS class with classList.toggle() to visually mark a task as completed.
- To update a button's label dynamically to reflect a task's completed state.
- To remove an element from the DOM using the remove() method.
- To conditionally show or hide an empty-state message based on the task count.
- To apply CSS3 Flexbox styling and a gradient background for a clean, card-based task list layout.

4. THEORY

This program is event-driven: nothing happens until the user clicks a button. Each task the user adds is represented as a task card (a <div class="task-item">) built entirely with JavaScript at runtime, rather than written directly in the HTML.

Every task card is created with its own Complete and Delete buttons, and each button's click listener is defined inside the same event handler that created the card. This means the listener automatically "remembers" which specific task card it belongs to — a JavaScript concept known as a closure — so clicking Delete on one task never affects another.

The key methods and concepts used in this program are:

- `document.createElement("div"/"span"/"button")` creates a new, empty HTML element in memory for each part of a task card.
- `appendChild()` attaches a created element as a child of another element, building the card and then attaching it to the visible list.
- `addEventListener("click", fn)` runs a function whenever a button is clicked; used for the Add, Complete, and Delete buttons.
- `classList.toggle("completed")` adds the completed class if it is absent and removes it if present, switching the strikethrough style on and off.
- `classList.contains("completed")` checks the task's current state so the Complete button's label can be switched between "Complete" and "Completed".
- `element.remove()` detaches a specific task card from its parent container, deleting the task from the page.
- `.trim()` strips leading and trailing whitespace so a task made only of spaces is treated as empty.
- `style.display` toggles the visibility of the "No tasks available." message based on how many task cards remain in the list.

The basic flow of the page is:

User types a task and clicks Add Task → the click handler runs → the input is validated → a task card with the task text, a Complete button, and a Delete button is built with DOM methods → the card is appended to the list → the input is cleared → the empty-state message is shown or hidden → Complete toggles a class and its own label on that card; Delete removes the card from the DOM.

5. ALGORITHM / PROCEDURE

- Create three files: `index.html`, `style.css` and `script.js`, and link `style.css` in the head and `script.js` at the end of the body.
- Add a page header, a text input for the task name, an "Add Task" button, an empty container (`<div id="taskList">`) for the task cards, and an empty-state `<p>` message.
- In `script.js`, select the input, button, list container, and empty-message elements using `document.getElementById()`.
- Define an `updateEmptyMessage()` function that shows the empty-state message only when the list container has no children.
- Attach a click listener to the Add Task button that reads and trims the input value, and alerts the user if it is blank.
- Inside the handler, create a task container `div`, a `<span>` for the task text, and a Complete and a Delete button, and set the text/labels.
- Attach a click listener to the Complete button that toggles a "completed" class on the task text and switches its own label between "Complete" and "Completed".
- Attach a click listener to the Delete button that removes its own task card from the list using `remove()` and calls `updateEmptyMessage()`.
- Append the text and both buttons to the task card, and append the card to the list container.
- Clear the input field and call `updateEmptyMessage()` after a task is added.
- Call `updateEmptyMessage()` once on page load so the empty message is correct before any task is added.
- Save the files and open `index.html` in a browser to test adding, completing, and deleting tasks.

6. PROGRAM

index.html

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6
7    <title>My To-Do List</title>
8
9    <link rel="stylesheet" href="style.css">
10 </head>
11
12 <body>
13
14   <div class="container">
15
16     <!-- Page heading -->
17     <header>
18       <h1>My To-Do List</h1>
19       <p>Organize your tasks and stay productive</p>
20     </header>
21
22     <!-- Section for entering a new task -->
23     <div class="task-input">
24
25       <input
26         type="text"
27         id="taskInput"
28         placeholder="Enter a task..."
29       >
30
31       <button id="addTaskBtn">
32         Add Task
33       </button>
34
35     </div>
36
37     <!-- Tasks will be added dynamically here -->
38     <div id="taskList"></div>
39
40     <!-- Message shown when there are no tasks -->
41     <p id="emptyMessage">
42       No tasks available.
43     </p>
44
45   </div>
46
47   <script src="script.js"></script>
48
49 </body>
50 </html>
51
```

style.css

```
1  /* Remove default spacing */
2  * {
3    margin: 0;
4    padding: 0;
```

```
5     box-sizing: border-box;
6 }
7
8 /* Main page styling */
9 body {
10     font-family: Arial, sans-serif;
11     min-height: 100vh;
12
13     background: linear-gradient(
14         135deg,
15         #667eea,
16         #764ba2
17     );
18
19     padding: 40px 20px;
20 }
21
22 /* Main container */
23 .container {
24     max-width: 700px;
25     margin: auto;
26 }
27
28 /* Page heading */
29 header {
30     text-align: center;
31     color: white;
32     margin-bottom: 25px;
33 }
34
35 header h1 {
36     font-size: 34px;
37     margin-bottom: 8px;
38 }
39
40 header p {
41     font-size: 15px;
42 }
43
44 /* Input section */
45 .task-input {
46     display: flex;
47     gap: 10px;
48
49     background: white;
50     padding: 20px;
51
52     border-radius: 16px;
53
54     box-shadow: 0 10px 25px
55         rgba(0, 0, 0, 0.2);
56
57     margin-bottom: 20px;
58 }
59
60 /* Task input box */
61 #taskInput {
62     flex: 1;
63
64     padding: 13px;
65 }
```

```
66     border: 1px solid #ccc;
67     border-radius: 8px;
68
69     font-size: 15px;
70 }
71
72 /* Add button */
73 #addTaskBtn {
74     padding: 13px 22px;
75
76     border: none;
77     border-radius: 8px;
78
79     background: #667eea;
80     color: white;
81
82     font-size: 15px;
83     font-weight: bold;
84
85     cursor: pointer;
86 }
87
88 /* Button hover effect */
89 #addTaskBtn:hover {
90     background: #5369d8;
91     transform: translateY(-2px);
92 }
93
94 /* Individual task */
95 .task-item {
96     display: flex;
97     align-items: center;
98     gap: 12px;
99
100     background: white;
101
102     padding: 16px;
103     margin-bottom: 12px;
104
105     border-radius: 12px;
106
107     box-shadow: 0 5px 15px
108         rgba(0, 0, 0, 0.15);
109 }
110
111 /* Task text */
112 .task-text {
113     flex: 1;
114
115     font-size: 16px;
116     font-weight: 500;
117 }
118
119 /* Buttons inside task */
120 .complete-btn,
121 .delete-btn {
122     border: none;
123     border-radius: 7px;
124
125     padding: 9px 13px;
126 }
```

```

127     color: white;
128     font-weight: bold;
129
130     cursor: pointer;
131 }
132
133 /* Complete button */
134 .complete-btn {
135     background: #28a745;
136 }
137
138 /* Delete button */
139 .delete-btn {
140     background: #dc3545;
141 }
142
143 /* Hover effects */
144 .complete-btn:hover {
145     background: #218838;
146 }
147
148 .delete-btn:hover {
149     background: #c82333;
150 }
151
152 /* Completed task */
153 .completed {
154     text-decoration: line-through;
155     opacity: 0.6;
156 }
157
158 /* Empty list message */
159 #emptyMessage {
160     text-align: center;
161
162     color: white;
163
164     font-size: 16px;
165
166     margin-top: 25px;
167 }
168

```

script.js

```

1  // Select the required HTML elements
2  const taskInput = document.getElementById("taskInput");
3  const addTaskBtn = document.getElementById("addTaskBtn");
4  const taskList = document.getElementById("taskList");
5  const emptyMessage = document.getElementById("emptyMessage");
6
7
8  // Function to update the empty-list message
9  function updateEmptyMessage() {
10
11     if (taskList.children.length === 0) {
12         emptyMessage.style.display = "block";
13     } else {
14         emptyMessage.style.display = "none";
15     }
16 }
17

```

```

15     }
16 }
17
18
19 // Add a new task when the button is clicked
20 addTaskBtn.addEventListener("click", function () {
21
22     // Get the task entered by the user
23     const taskText = taskInput.value.trim();
24
25     // Prevent adding an empty task
26     if (taskText === "") {
27         alert("Please enter a task.");
28         return;
29     }
30
31
32     // Create a new task container
33     const taskItem = document.createElement("div");
34     taskItem.className = "task-item";
35
36
37     // Create the task text
38     const task = document.createElement("span");
39     task.className = "task-text";
40
41     task.textContent = taskText;
42
43
44     // Create Complete button
45     const completeBtn =
46         document.createElement("button");
47
48     completeBtn.className = "complete-btn";
49     completeBtn.textContent = "Complete";
50
51
52     // Complete button event
53     completeBtn.addEventListener("click", function () {
54
55         // Add/remove completed styling
56         task.classList.toggle("completed");
57
58         // Change button text
59         if (task.classList.contains("completed")) {
60             completeBtn.textContent = "Completed";
61         } else {
62             completeBtn.textContent = "Complete";
63         }
64     });
65
66
67     // Create Delete button
68     const deleteBtn =
69         document.createElement("button");
70
71     deleteBtn.className = "delete-btn";
72     deleteBtn.textContent = "Delete";
73
74
75     // Delete button event

```

```

76     deleteBtn.addEventListener("click", function () {
77
78         // Remove the task from the webpage
79         taskItem.remove();
80
81         // Show message if no tasks remain
82         updateEmptyMessage();
83     });
84
85
86     // Add all elements to the task container
87     taskItem.appendChild(task);
88     taskItem.appendChild(completeBtn);
89     taskItem.appendChild(deleteBtn);
90
91     // Add the task to the webpage
92     taskList.appendChild(taskItem);
93
94
95     // Clear the input box
96     taskInput.value = "";
97
98     // Update empty message
99     updateEmptyMessage();
100 });
101
102
103 // Display the empty message when the page loads
104 updateEmptyMessage();
105

```

7. CODE EXPLANATION

Code	Explanation
document.getElementById()	Selects a specific element from the page by its id attribute.
addEventListener("click", fn)	Runs the given function whenever the Add, Complete, or Delete button is clicked.
.value / .trim()	Reads the current text typed into the input field and strips extra whitespace.
document.createElement()	Creates a new, empty HTML element (div, span, button) in memory.
element.textContent	Sets the visible text inside an element, such as the task name or button label.
appendChild()	Attaches a created element as a child of another element in the DOM.
classList.toggle("completed")	Adds or removes the completed class on a task, switching the strikethrough style.
classList.contains("completed")	Checks whether a task is currently marked complete, used to update the button label.
element.remove()	Detaches a specific task card from the list, deleting that task from the page.

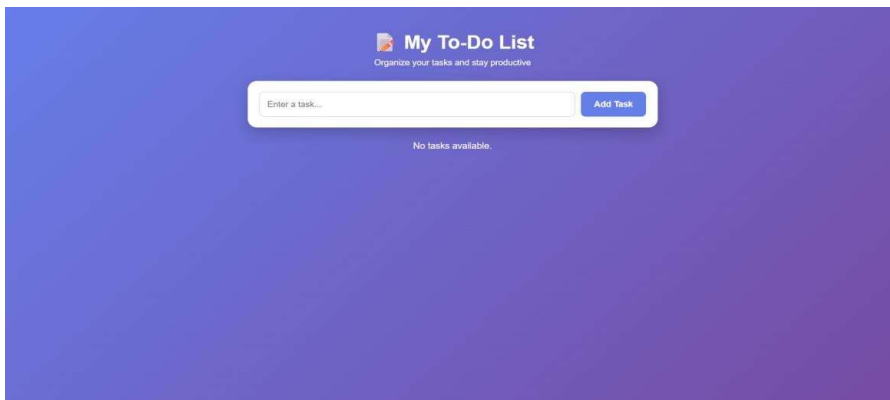
Code	Explanation
style.display	Shows or hides the "No tasks available." message depending on the task count.

8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and the app was tested with different inputs and actions; the results were recorded below.

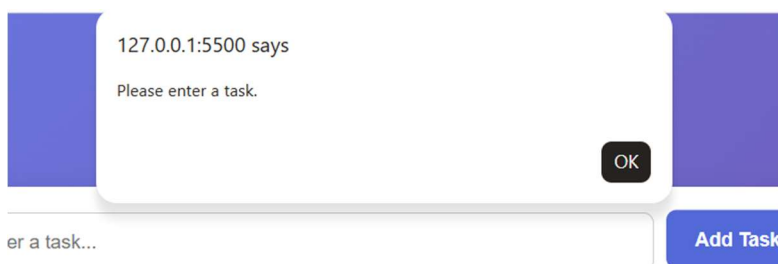
Test Case: TC-01
Action: Open index.html in a web browser.
Expected Result: Empty task list is shown with the "No tasks available." message centered below the input card.
Actual Result: Empty state displayed correctly on initial load.
Status: Pass

TC-01 Output:



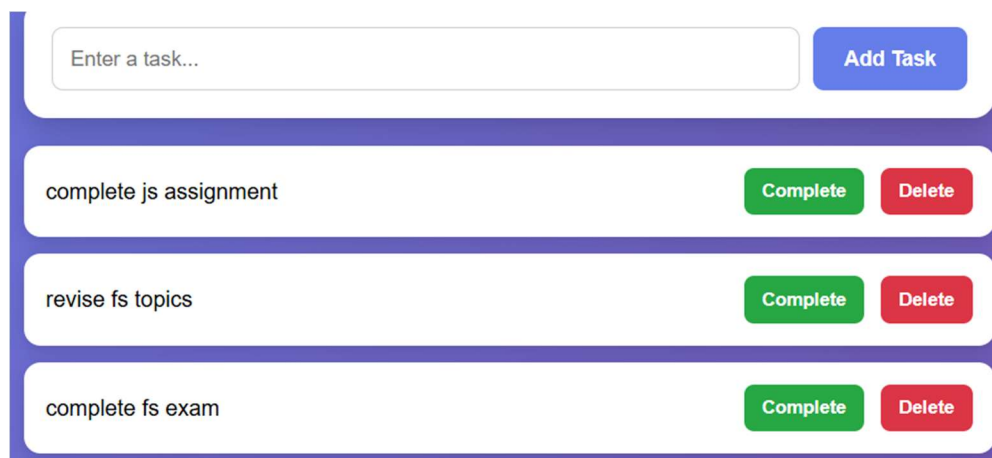
Test Case: TC-02
Action: Click "Add Task" with the input field left completely empty or containing only spaces.
Expected Result: An alert prompts "Please enter a task." and script returns early without creating a task card.
Actual Result: Alert shown as expected; no blank item added to DOM.
Status: Pass

TC-02 Output:



Test Case: TC-03
Action: Enter task names (e.g., "complete js assignment", "revise fs topics", "complete fs exam") and click "Add Task".
Expected Result: Tasks appear as dynamic cards with "Complete" and "Delete" buttons, input clears, and "No tasks available." message hides.
Actual Result: Task cards rendered correctly; empty state hidden.
Status: Pass

TC-03 Output:



Enter a task... Add Task

complete js assignment Complete Delete

revise fs topics Complete Delete

complete fs exam Complete Delete

Test Case: TC-04
Action: Click the "Complete" button on a specific task (e.g., "complete js assignment").
Expected Result: Strikethrough style and opacity reduction apply to the task text, and button label switches to "Completed".
Actual Result: .completed class toggled on and button label updated.
Status: Pass

TC-04 Output:



~~complete js assignment~~ Completed Delete

Test Case: TC-05
Action: Click the "Delete" button on a task item.
Expected Result: The targeted task card is removed from the DOM using .remove() without affecting other tasks.
Actual Result: Task card removed immediately.
Status: Pass

TC-05 Output:



revise fs topics Complete Delete

complete fs exam Complete Delete

Test Case: TC-06

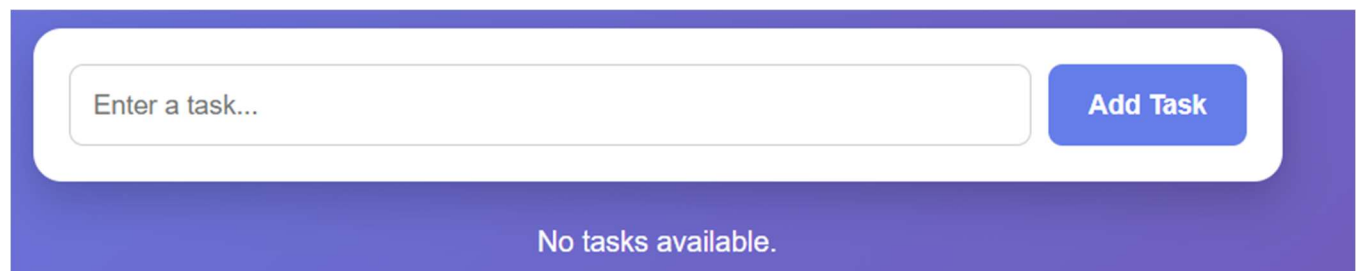
Action: Click "Delete" on all remaining tasks until the list container is empty.

Expected Result: `taskList.children.length === 0` evaluates to true, triggering `emptyMessage.style.display = "block"`.

Actual Result: Empty state message reappeared automatically.

Status: Pass

TC-06 Output:

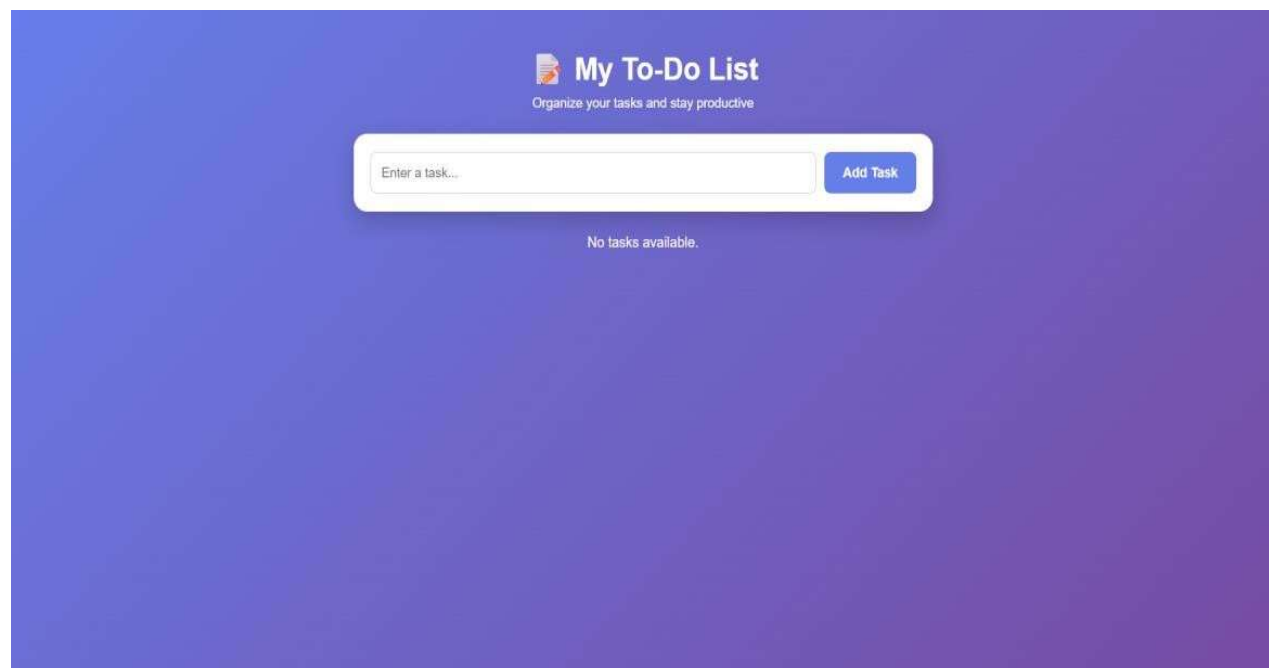


Enter a task... Add Task

No tasks available.

9. OUTPUT

Output 1: Empty To-Do List

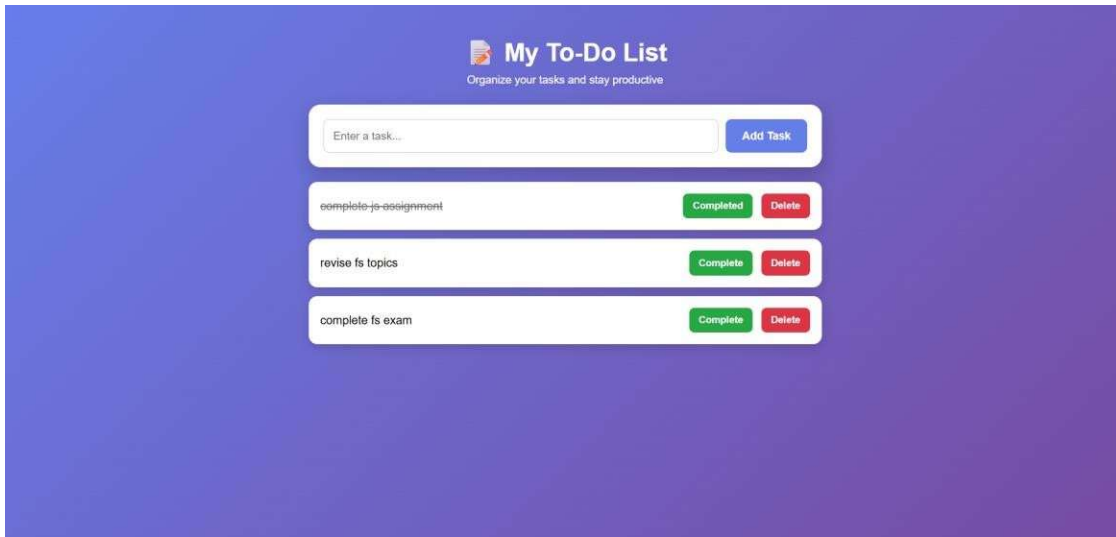


 **My To-Do List**
Organize your tasks and stay productive

Enter a task... Add Task

No tasks available.

Output 2: Tasks Added, One Marked Complete



10. RESULT

Thus, an interactive to-do list application was successfully built using JavaScript DOM manipulation. Adding, completing, and deleting tasks all worked correctly without reloading the page, and the empty-state message updated correctly as tasks were added and removed. The Complete button's label dynamically switches to "Completed" once a task is marked done and reverts to "Complete" when it is undone, making the toggle behaviour clear to the user.